

ISTANBUL TECHNICAL UNIVERSITY
COMPUTER ENGINEERING DEPARTMENT

BLG 222E
COMPUTER ORGANIZATION
PROJECT 1 REPORT

CRN : 21335

LECTURER : Deniz Turgay Altılar

GROUP MEMBERS:

150210925 : NEJRA GUTIC (Group Representative)

150220042 : ZEYNEP NUR YILMAZ

SPRING 2024

Contents

1	INTRODUCTION	1
2	MATERIALS AND METHODS	1
2.1	PART 1	1
2.2	PART 2	2
2.2.1	2.a	2
2.2.2	2.b	2
2.2.3	2.c	3
2.3	PART 3	4
2.4	PART 4	5
3	RESULTS	7
4	DISCUSSION	9
5	CONCLUSION	9

1 INTRODUCTION

In this project, we were required to design and implement registers, register files and an arithmetic logic unit as modules using Verilog hardware description language to select from and perform various operations on given input values. We were also expected to simulate these implementations using provided simulation files. The project consisted of 4 parts.

Parts 1, 2.a, 2.b and 3 were done by Nejra Gutic and parts 2.c and 4 were done by Zeynep Nur Yilmaz; though we were in contact for the most part and debugging was done together.

2 MATERIALS AND METHODS

2.1 PART 1

In the first part of the project, we implemented a 16-bit register with 8 functionalities that are controlled by 3-bit control signals (FunSel) and an enable input (E). When the value of enable input is 0 register retains its previous value, otherwise value of the 16-bit Q output changes based on the value of FunSel input (as given in the table in Project file). On the photo below, the elaborated design of this register is provided:

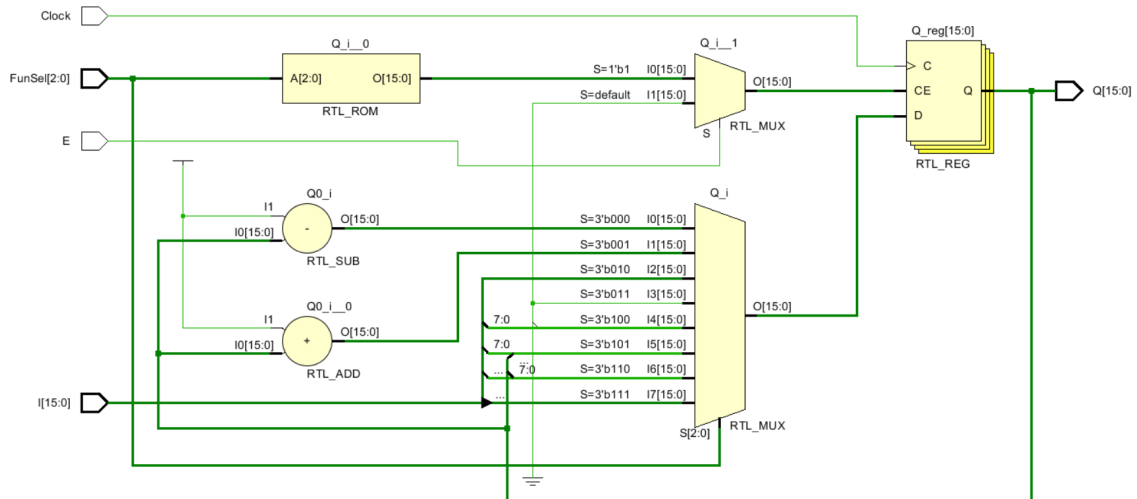


Figure 1: Register

2.2 PART 2

In the second part of this project, register file (structure containing many registers) was designed using register design from the part 1.

2.2.1 2.a

Firstly, 16-bit Instruction Register was designed. It contains 8-bit data input, LH and Write input. Write input decides whether data is loaded in register (Write=0 means that the previous value is retained). Furthermore, because data input of IR is only 8-bit while register itself can store 16-bit data, input LH is used to decide which part of register is loaded (either higher (15:8) or lower part (7:0)). Output of IR is 16-bit IROut.

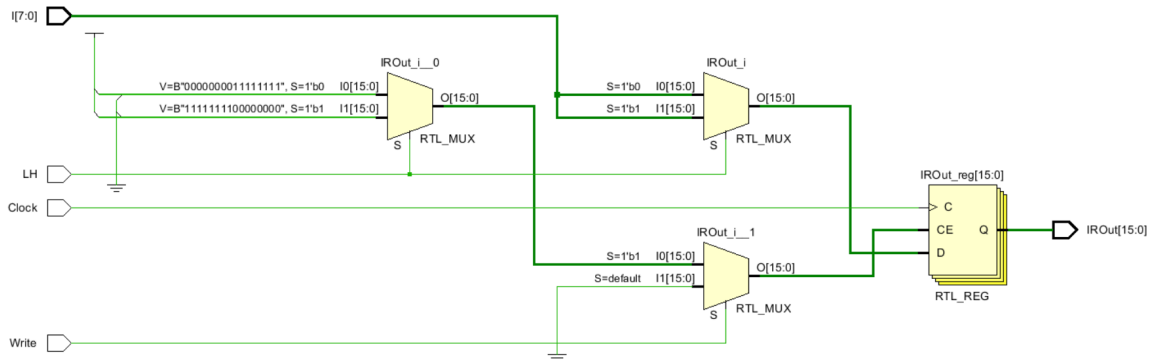


Figure 2: Instruction Register

2.2.2 2.b

In this part, design containing four 16-bit general purpose registers and four 16-bit scratch registers is implemented. Also, it has 16-bit data input and two 16-bit data outputs (OutA and OutB). To implement these 16-bit registers module register from the first part of this project was used. 3-bit FunSel input decides which function will be done (based on the table provided in the file). Next, 4-bit inputs RegSel and ScrSel are deciding which of the general purpose and scratch registers are enabled at the moment. Finally, 3-bit inputs OutASel and OutBSel are feeding outputs OutA and OutB, so deciding which of the registers output value will be shown on the OutA and OutB.

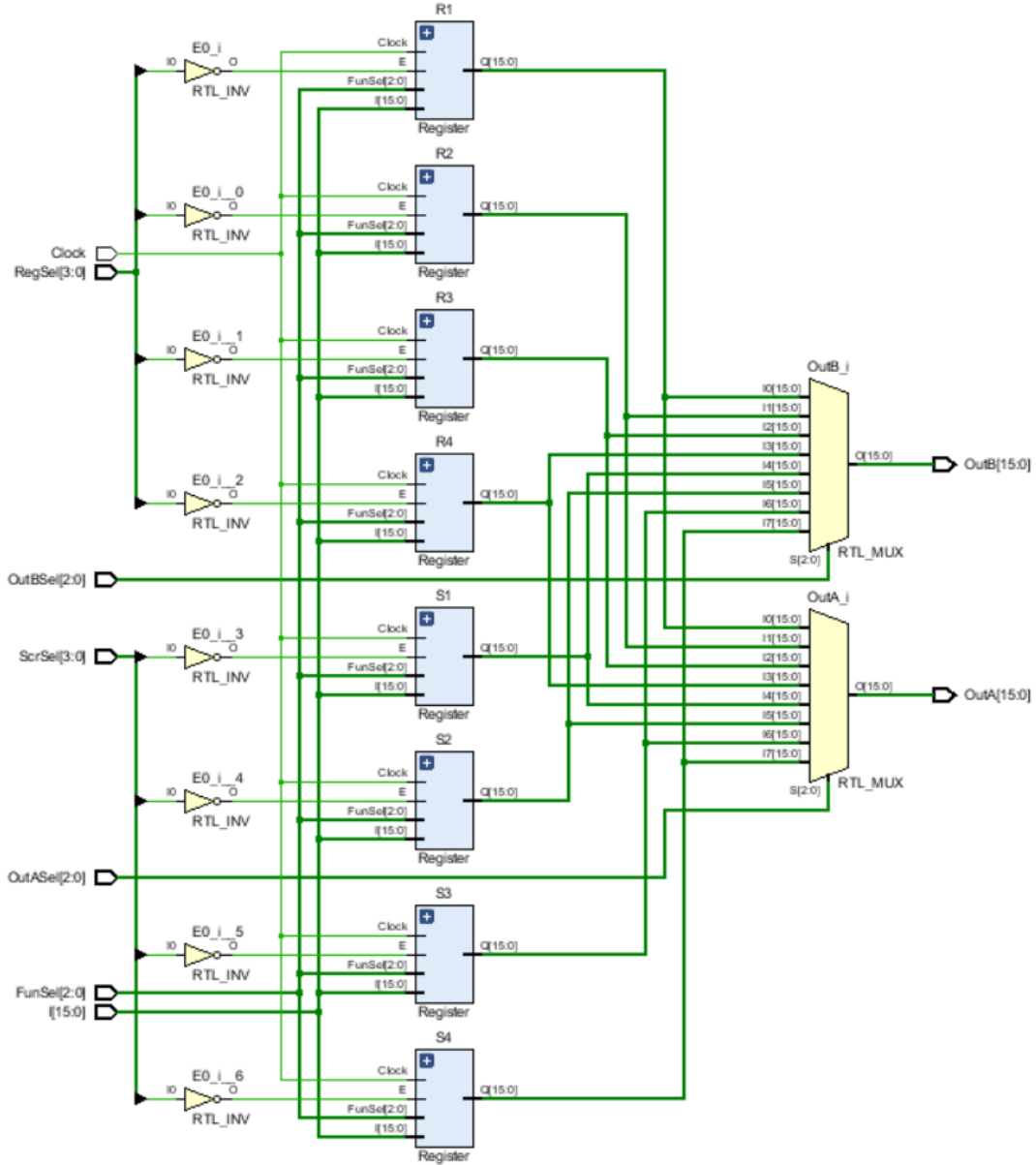


Figure 3: Register File

2.2.3 2.c

In the last part of the second part of the project, Address Register File (ARF) design was implemented. Again, register design from the part 1 was used to implement 3 registers: program counter (PC), address register (AR), and stack pointer (SP). Rules for the 2-bit inputs OutCSel and OutDSel are the same as in the previous part, they are deciding which registers output will be feeding 16-bit outputs OutC and OutD. 3-bit inputs FunSel and RegSel are behaving the same as in the previous part, as well.

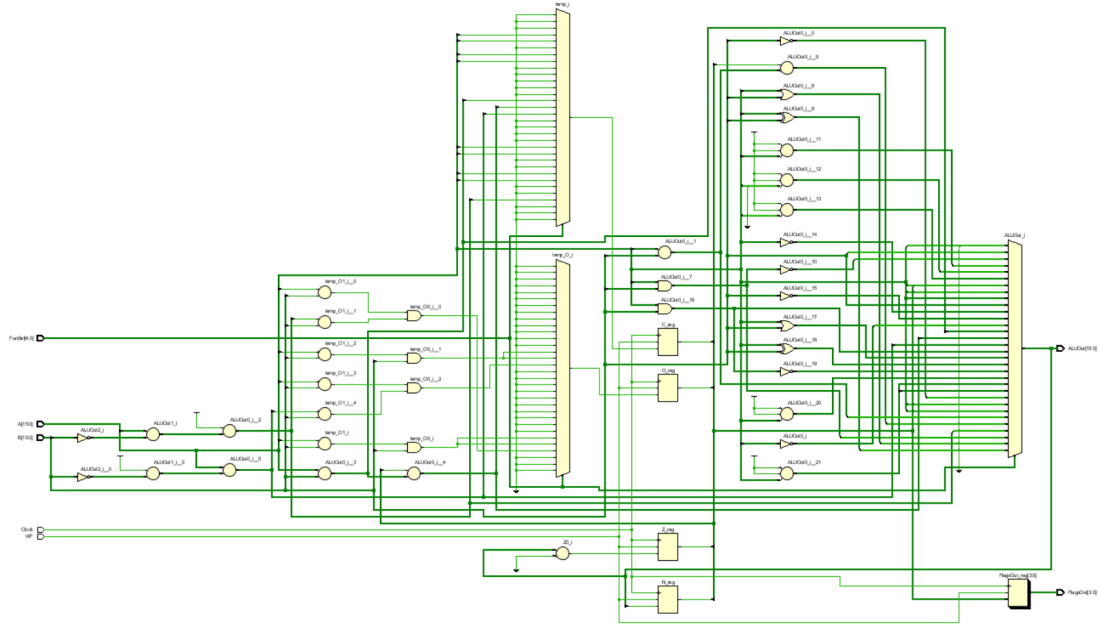


Figure 5: Arithmetic Logic Unit

2.4 PART 4

In the last part of the project, we were required to implement a system that combines everything from the previous parts. The system consisted of an instruction register, a register file, an address register file and an arithmetic logic unit with the addition of a memory and 3 multiplexers. Value selection tables for the multiplexers are provided in the project file and the elaborated design of the whole system can be seen below.

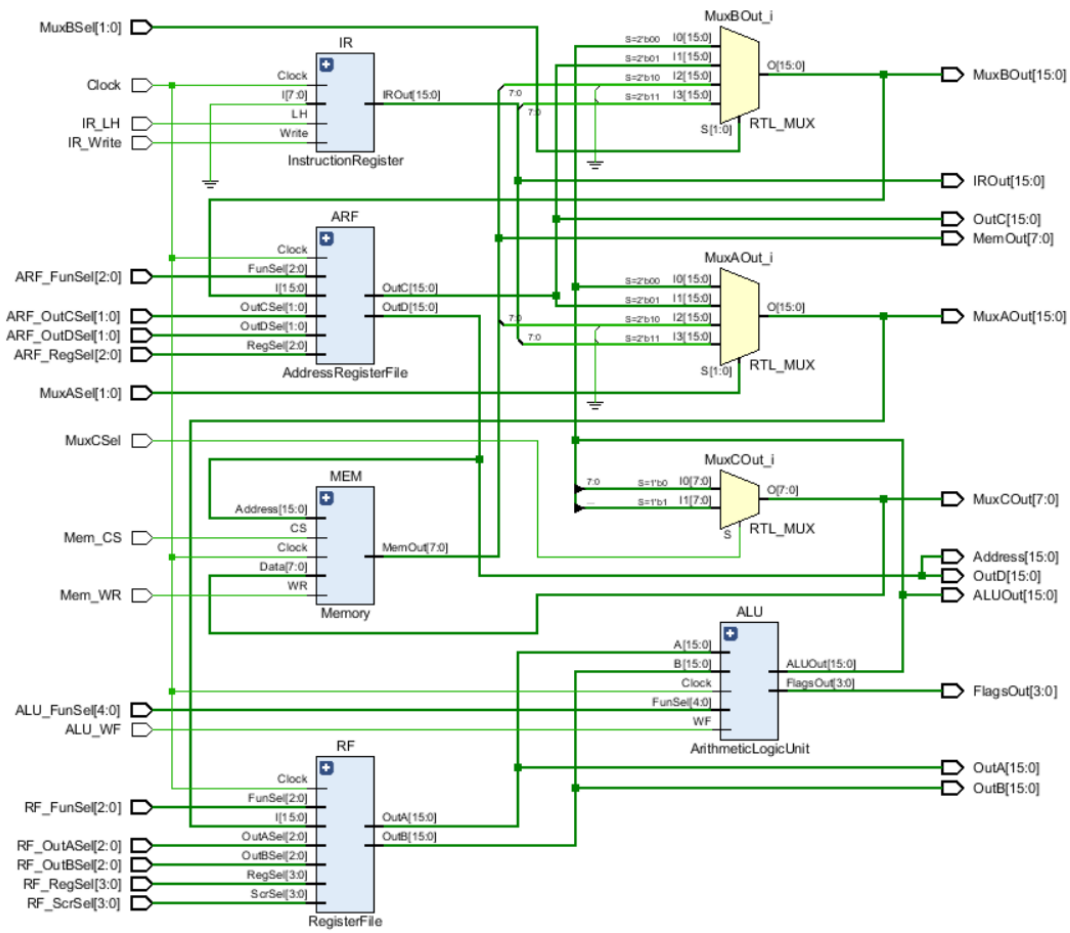


Figure 6: Arithmetic Logic Unit System

3 RESULTS

The simulation results of our implementations are given below.

Part 1:

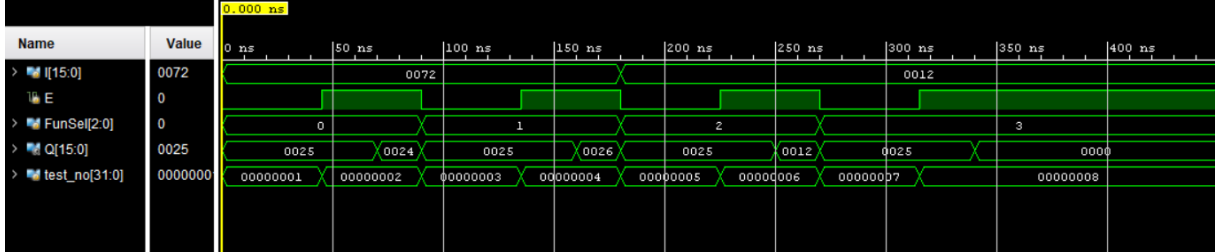


Figure 7: Register Simulation

Part 2.a:

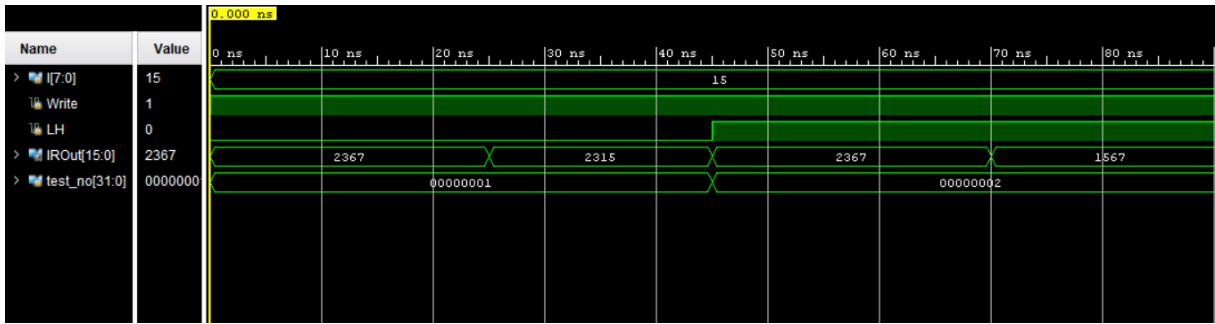


Figure 8: Instruction Register Simulation

Part 2.b:

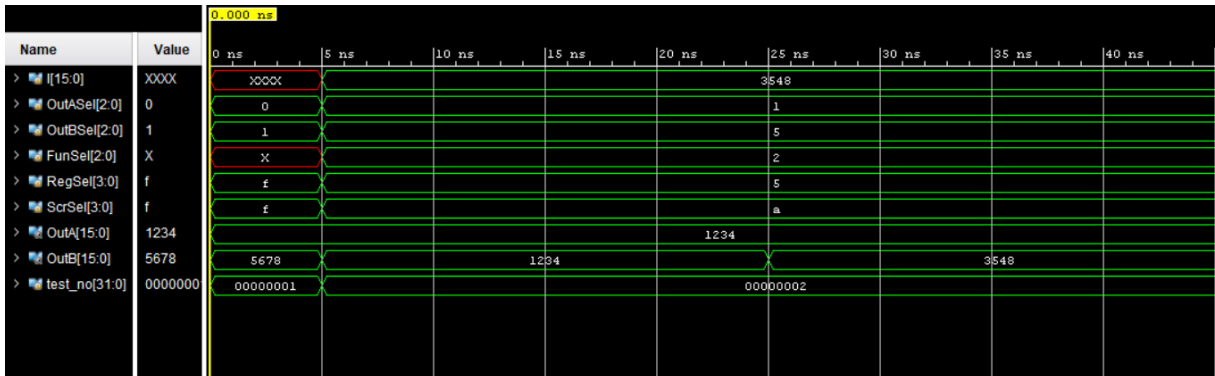


Figure 9: Register File Simulation

Part 2.c:

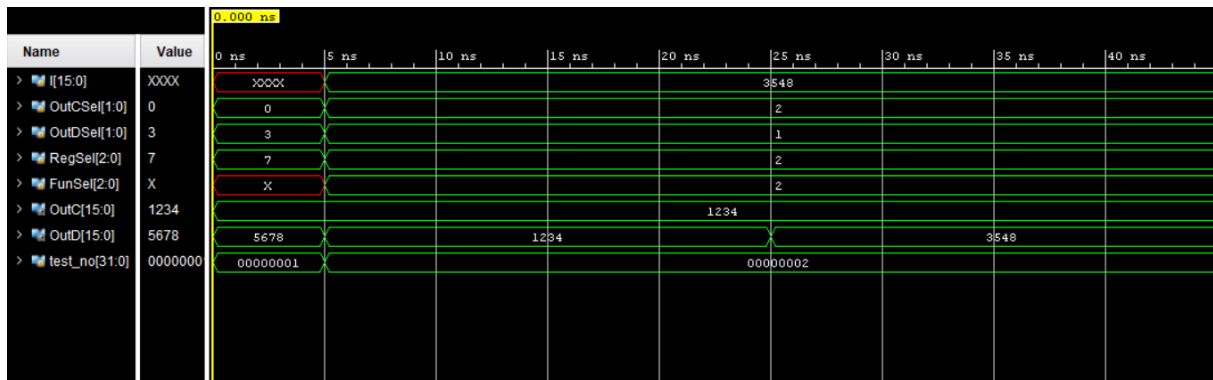


Figure 10: Address Register File Simulation

Part 3:

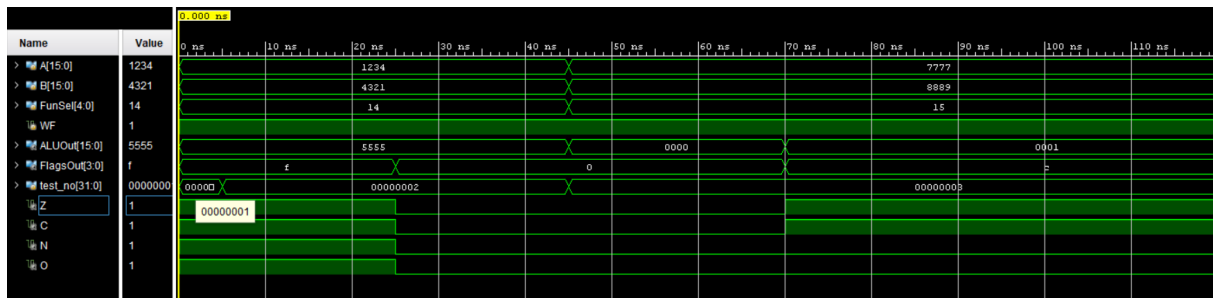


Figure 11: Arithmetic Logic Unit Simulation

Part 4:

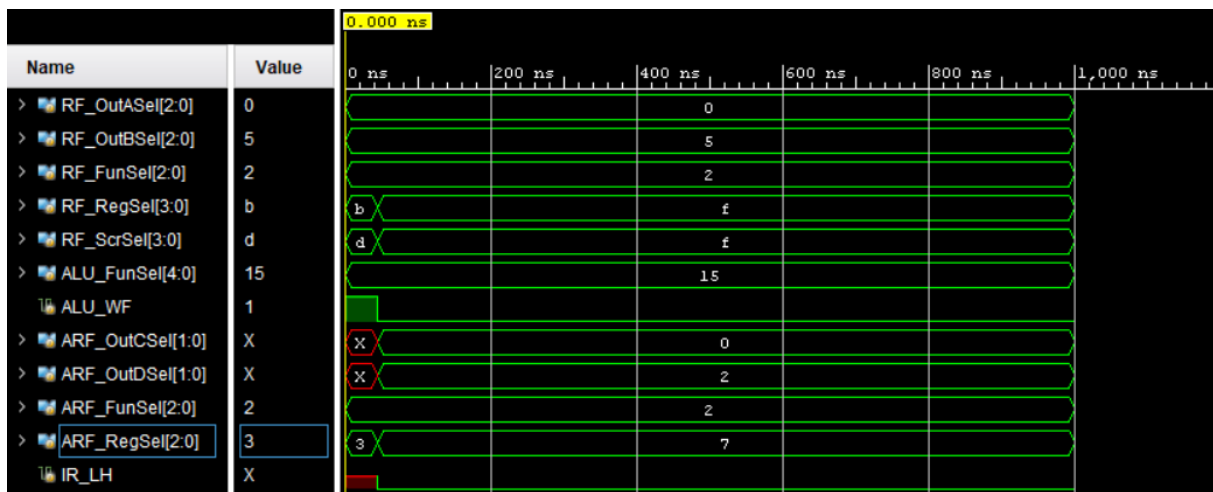


Figure 12: Arithmetic Logic Unit System Simulation

4 DISCUSSION

To start with, for this project we have been asked to implement Register, Register File, Instruction Register, Address Register and Arithmetic Logic Unit. As a final product, we have ended up with Arithmetic Logic Unit System. All of part mentioned were based on the simpler modules created before them. So that, for example, in Register Files, we have to used Register module, to implement all general purpose and scratch registers. Same thing applies for Address Register, where we used Register for implementing PC, AR, and SP registers. In the end, for our final task, constructing whole ALU System, we have used all modules created before: Register File, ALU, Address Register File, Instruction register. Apart from this, Memory module was used as well, but this module was give in Project files, so there was no need for us to write it. Our only task was to interconnect it with other parts and understand its purpose within our system. Besides, three multiplexers were used, two of which being 4to1 and one being 2to1. Their purpose was to decide, based on their selection inputs, which data is being forwarded to unit they are connected to. That way, MUX A decides which among ALU output, ARF output, memory output, and lower level of IR register, will be fed to Register File. Same thing, considering inputs, is valid for MUX B as well, only it is feeding ARF's input. Lastly, 2to1 multiplexer, MUX C is deciding wheter lower or higher seven bits of ALU output will be sent to the memory input. By applying all of this, we have ended up with complete ALU system that can perform both arithmetic and logical operations based on the selection inputs in the system, also storing ALU output values in memory, making it possible to use it for following operations.

5 CONCLUSION

When we first started doing this project, our main problem was understanding how we were supposed to use Verilog to test our implementations. After a few trial and error we were able to overcome this struggle. The first 2 parts of the project were quite uncomplicated compared to the parts 3 and 4. In part 3, there were many cases to be implemented and this was a bit time-consuming. Also, particularly confusing were the 8-bit cases in which it was pretty unclear what should happen with 8 bits that are not used as the output. And in the last part, understanding the connections between the inputs and outputs of different modules in the system and the overall implementation was quite confusing. But we were still able to finish all of the implementations and the test results were successful.